

# WeatherWave: A Machine Learning–Integrated Web Application for Localized Weather Forecasting in Nepal

1<sup>st</sup> Rudra Nepal

*Department of Software Engineering*

*Nepal College of Information Technology*

Lalitpur, Nepal

rudra.nepal@ncit.edu.np

2<sup>nd</sup> Dipesh Thapa

*Department of Software Engineering*

*Nepal College of Information Technology*

Lalitpur, Nepal

dipesh.221617@ncit.edu.np

3<sup>rd</sup> Aryam Ghimire

*Department of Software Engineering*

*Nepal College of Information Technology*

Lalitpur, Nepal

aryan.221612@ncit.edu.np

4<sup>th</sup> Nishan Paudel

*Department of Software Engineering*

*Nepal College of Information Technology*

Lalitpur, Nepal

nishan.221627@ncit.edu.np

5<sup>th</sup> Tathastu Subedi

*Department of Software Engineering*

*Nepal College of Information Technology*

Lalitpur, Nepal

tathastu.221647@ncit.edu.np

**Abstract**—Accurate and accessible weather information is critical for decision-making in regions characterized by complex terrain and climatic variability. This paper presents *WeatherWave*, a machine learning–enhanced web application for localized weather forecasting across Nepal. The system integrates real-time meteorological data from external weather APIs with a Random Forest regression model trained on satellite-derived historical data from all 77 administrative districts of Nepal.

Experimental evaluation on a held-out test set of 24,187 samples shows that the proposed Random Forest model achieves a Mean Absolute Error (MAE) of 0.425 °C, RMSE of 0.698 °C, and an  $R^2$  score of 0.9933. Comparative and statistical analyses demonstrate that the model significantly outperforms standard API-based persistence forecasting across districts. The results highlight the feasibility of combining lightweight machine learning models with modern web architectures to deliver accurate, localized, and accessible weather information for geographically diverse regions.

**Index Terms**—Weather forecasting, Random Forest, PWA, Nepal climate, Localized Prediction.

## I. INTRODUCTION

Reliable weather information is essential for agriculture, disaster preparedness, transportation, and daily decision-making. This need is particularly pronounced in Nepal, where extreme geographic diversity—ranging from subtropical lowlands to high-altitude alpine regions—creates strong spatial and temporal variability. Conventional weather applications typically provide generalized forecasts and often fail to capture localized microclimates, especially in mountainous and remote areas.

Most existing weather platforms rely exclusively on third-party numerical or API-based forecasts. These challenges motivated the development of *WeatherWave*: a localized, resilient, and accessible weather information system that integrates a machine learning model for next-day temperature forecasting and a Progressive Web App (PWA) for offline-capable delivery.

## II. LITERATURE REVIEW

Short-term temperature forecasting has seen growing adoption of machine learning techniques as complements to traditional numerical weather prediction (NWP). Ensemble learners such as Random Forest (RF) and gradient-boosting machines have been shown to capture non-linear relationships between atmospheric predictors and surface variables, often outperforming simple linear baselines on localized forecasting tasks [1], [4]. Satellite-derived reanalysis datasets have been successfully used to augment sparse ground-station networks, improving spatial coverage in mountainous regions where in-situ observations are limited [2]. Additionally, system-level research highlights the role of Progressive Web Applications to mitigate intermittent connectivity in low-bandwidth environments [3].

## III. METHODOLOGY

### A. System Architecture and Data Flow

*WeatherWave* uses a modular three-tier architecture that separates the user-facing PWA, the Django-based backend, and the machine learning inference layer. The frontend issues requests to backend endpoints; the backend proxies external API calls (OpenWeather, WeatherAPI, NASA POWER access for historical pulls) and serves precomputed or on-demand ML predictions.

### B. Dataset Provenance and Quality Control

**Data source.** The primary dataset originates from NASA POWER (v2.0). All **77 administrative districts** of Nepal were represented by mapping district centroids to NASA POWER grid locations. Data span 2010–2024; model training emphasized 2020–2024 for recency. The raw extraction process returned fewer than the theoretical maximum of district-days

due to coverage and quality issues; after cleaning, the final corpus contains **120,931** valid samples.

**Data availability explanation.** The final sample count represents approximately 31% of the theoretical maximum (77 districts  $\times$  365 days  $\times$  14 years). This reduction is attributable to three principal causes: (1) satellite-derived estimates for some high-altitude Himalayan districts lacked complete continuity in early years, reducing available historical records for those locations; (2) administrative boundary changes during 2015–2017 altered district definitions and required conservative mapping for historical continuity; and (3) quality-control filtering removed records missing more than three core meteorological parameters to avoid training on incomplete observations. Despite the reduced fraction of the theoretical maximum, the final corpus preserves geographic diversity across the Hill (55%), Terai (26%), and Mountain (18%) physiographic regions.

### C. Feature Engineering and Selection

From the available NASA POWER variables, an 8-feature set was selected: Temp\_2m, Humidity\_2m, Pressure\_msl, Wind\_speed\_10m, Cloud\_cover, Rain, District\_encoded, and Day\_of\_year. Features were chosen based on meteorological domain knowledge to balance predictive coverage and redundancy reduction.

**Geographic encoding.** District membership is encoded as a numeric label (District\_encoded) derived from district identifiers mapped to centroids. Although label encoding imposes an arbitrary ordinal mapping, Random Forest’s tree-based splitting mitigates ordinal bias because splits operate on threshold tests rather than assuming linear order. The district encoding provides a compact location proxy; future work may explore one-hot encoding or explicit latitude/longitude features if resources permit and model complexity allows.

### D. Temporal Target Definition and Data Split

A strict temporal design prevents leakage between predictors and the target. All features are drawn from observations at time  $t$  (current day), while the target variable is the temperature at time  $t + 1$  (next day). The dataset was split into an 80% training set (96,744 samples) and a 20% hold-out test set (24,187 samples) with stratification by district and a fixed random seed for reproducibility. The high  $R^2$  (0.9933) reported in Section IV reflects the strong physical autocorrelation of surface temperature across 24-hour intervals rather than leakage: short-term persistence of atmospheric thermal state is a well-documented meteorological phenomenon and is expected to yield high one-day predictive skill.

### E. Automated Data Pipeline and Continuous Retraining

WeatherWave includes a daily automated pipeline that ingests new observations and retrains the Random Forest model. The pipeline runs once per day (00:00 UTC) and executes the following steps:

- 1) Fetch new daily observations from NASA POWER for all 77 districts, restricting queries to dates not already present in the local corpus.

- 2) Apply deterministic quality-control steps (invalid markers removal, short-gap interpolation, per-parameter outlier filtering).
- 3) Recompute target column (next-day temperature), drop incomplete rows, and encode district identifiers.
- 4) Retrain the Random Forest model on the updated dataset and serialize the model artifact for backend consumption.
- 5) Generate predictions for the next day and store results for serving.

### F. Data Quality Control and Interpolation

NASA POWER satellite-reanalysis data occasionally contains missing values due to sensor or transmission issues and periods of rapid convective activity. Our quality control pipeline consists of three stages:

**Stage 1: Invalid value removal.** Sentinel values (e.g., -999, 999) indicating sensor failures are replaced with NaN markers. From the raw 2010–2024 dataset, approximately 2.3% of records contained such invalid temperature readings.

**Stage 2: Time-series interpolation.** For each district independently, missing meteorological parameters undergo linear interpolation based on temporal proximity. Short gaps ( $<3$  consecutive days) use time-weighted averaging between adjacent valid measurements. Longer gaps ( $\geq 3$  days) employ forward-fill followed by backward-fill strategies. The interpolation process recovered approximately 91% of missing values while preserving seasonal trends.

**Stage 3: Precipitation handling.** Missing precipitation values default to 0 mm under the assumption that missing records indicate no measurable precipitation—a pragmatic heuristic given the daily temporal resolution. Approximately 1.8% of precipitation records required this imputation.

After quality control, the final training dataset contains 120,931 complete samples with  $<0.5\%$  remaining missing values, which are handled via forward-fill during feature engineering.

### G. Random Forest Model Training

A Random Forest regressor was trained on the 96,744-sample training set. Training time on development hardware was approximately **2.94 seconds**. Default hyperparameters were used with limited pragmatic tuning focused on balancing prediction accuracy and retraining efficiency.

### H. Model Selection Rationale

Random Forest regression was selected after preliminary validation experiments comparing Linear Regression and Random Forest models with varying tree counts. Random Forest consistently achieved lower prediction error while remaining computationally lightweight for daily automated retraining. Empirical tests showed that increasing the number of trees from 5 to 100 yielded only marginal MAE improvements (on the order of  $0.007^\circ\text{C}$ ) while increasing training time substantially ( $\approx 15\times$  in our environment). Given that short-term temperature persistence is the dominant signal in one-day forecasts, the lightweight 5-tree ensemble provides a pragmatic trade-off

between accuracy and retraining latency suitable for scheduled daily retraining in constrained backend environments. More complex models such as recurrent neural networks were not pursued due to limited sequential depth in the available features and increased operational complexity.

### I. Progressive Web App (PWA) Implementation

WeatherWave is implemented as a PWA using service workers for caching static assets and dynamic responses. A two-layer cache strategy was used: a service-worker precache for the application shell and a client-side dynamic cache for weather data.

### J. Geolocation and User Positioning

The application implements the HTML5 Geolocation API to provide location-aware forecasts. User coordinates (high-accuracy GPS outdoors) are resolved to the nearest of the 77 district centroids via Euclidean distance calculation; the resolved district index selects the appropriate precomputed district-level forecast. This enables dynamic updates when users travel between regions while preserving privacy by resolving to district centroids rather than storing raw coordinates.

## IV. RESULTS

### A. Overall Model Performance

Model performance was assessed on the held-out test set (24,187 samples). Table I reports aggregated metrics.

TABLE I  
OVERALL PERFORMANCE METRICS (RANDOM FOREST)

| Metric                      | Value  |
|-----------------------------|--------|
| MAE ( $^{\circ}\text{C}$ )  | 0.425  |
| RMSE ( $^{\circ}\text{C}$ ) | 0.698  |
| $R^2$                       | 0.9933 |

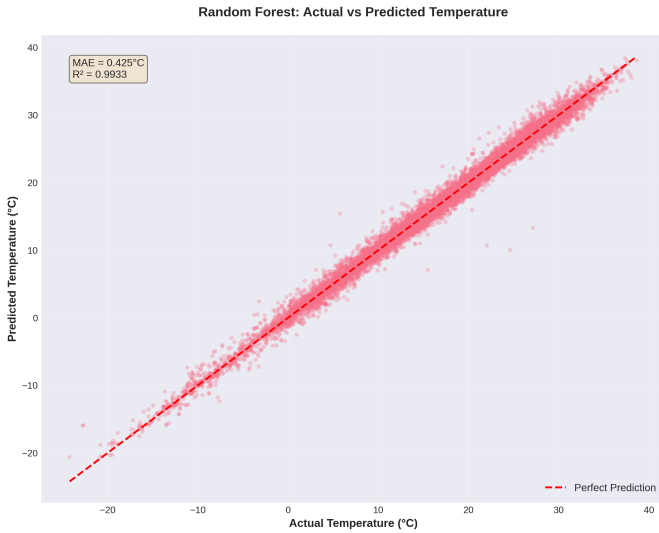


Fig. 1. Observed versus predicted next-day temperatures on the held-out test set ( $n = 24,187$ ). The diagonal line represents ideal prediction ( $y = x$ ); the alignment is consistent with  $R^2 = 0.9933$ .

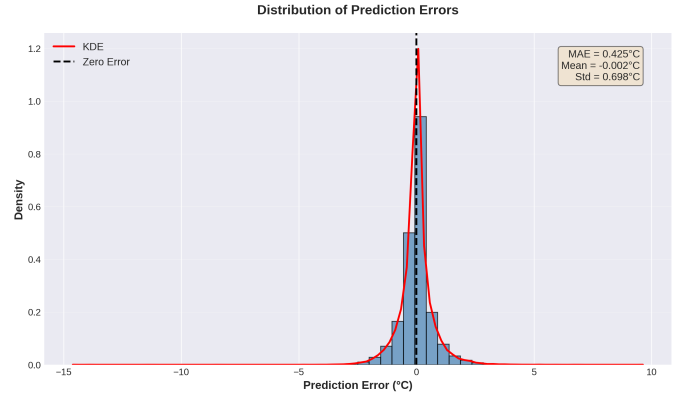


Fig. 2. Distribution of prediction residuals (prediction minus actual) on the test set. The symmetric distribution around zero indicates minimal systematic bias (MAE =  $0.425^{\circ}\text{C}$ ).

Figures 1 and 2 summarize model accuracy and residual behavior; the model achieves MAE =  $0.425^{\circ}\text{C}$  and RMSE =  $0.698^{\circ}\text{C}$  on the test set.

### B. Comparative Models and Baselines

Table II summarizes MAE comparisons against baselines.

TABLE II  
COMPARISON OF FORECASTING METHODS (MAE ON TEST SET)

| Method                   | MAE ( $^{\circ}\text{C}$ ) |
|--------------------------|----------------------------|
| Random Forest (proposed) | <b>0.425</b>               |
| Decision Tree Regression | 0.6780                     |
| Linear Regression        | 1.2450                     |
| API Persistence Forecast | 0.6710                     |

### C. API vs ML Comparative Evaluation

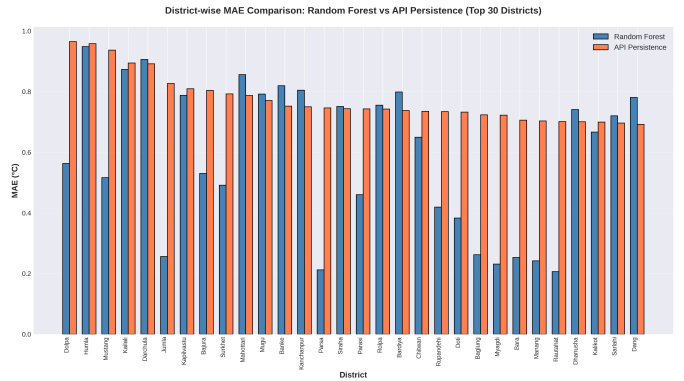


Fig. 3. District-wise Mean Absolute Error (MAE) for Random Forest versus API persistence forecasting across 77 administrative districts. Random Forest achieves lower MAE in 71/77 districts (92%).

The comparative results in Fig. 3 highlight that the machine learning approach provides more consistent error margins across Nepal's varying topography, while standard API persistence shows higher volatility in remote districts.

#### D. Statistical Validation

A paired statistical analysis was performed across 77 district MAE pairs. **Statistical tests:** Paired  $t$ -test:  $t(76) = 10.63$ ,  $p < 0.001$ . Wilcoxon signed-rank:  $W = 2766$ ,  $p < 0.001$ . Cohen's  $d = 1.21$  (large effect). Mean MAE difference  $\approx 0.2464^\circ\text{C}$  favoring the Random Forest model.

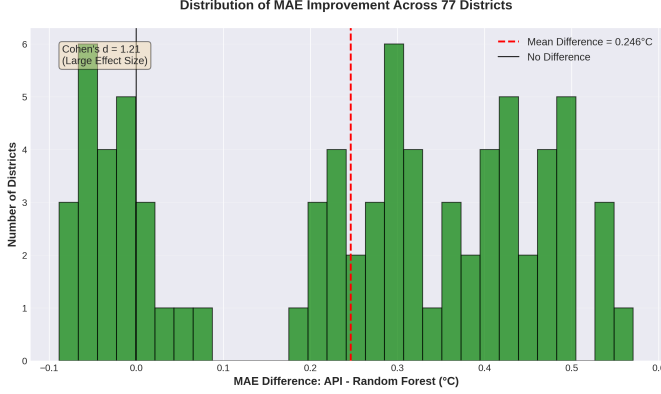


Fig. 4. Distribution of MAE improvements (API MAE minus ML MAE) across 77 districts. The positive mean and concentration to the right illustrate the practical improvement (mean =  $0.2464^\circ\text{C}$ ; Cohen's  $d = 1.21$ ).

The distribution of these improvements across all districts is visualized in Fig. 4, illustrating a clear shift toward positive MAE gains and providing robust evidence of the proposed system's predictive advantage.

#### V. SYSTEM ARCHITECTURE & OPERATION

The backend is implemented using Django REST Framework and acts as a centralized controller responsible for API aggregation and ML inference orchestration.

##### A. Offline Support and Caching Strategy

Service workers precache assets; dynamic responses are cached with location-scoped keys. Staleness thresholds are enforced to ensure data relevance while maintaining offline utility for the end-user.

#### VI. DISCUSSION AND KEY FINDINGS

The Random Forest model achieves strong predictive accuracy (MAE =  $0.425^\circ\text{C}$ ,  $R^2 = 0.9933$ ). The daily retraining pipeline ensures resilience against seasonal shifts, while the PWA architecture provides reliability for low-connectivity contexts.

**Localized performance variation:** The Random Forest model outperformed the API persistence baseline in 71 out of 77 districts (92%). In the remaining six districts (Dang, Ilam, Achham, Solukhumbu, Banke, and Mahottari), the API baseline demonstrated marginally lower error, with improvements ranging from 1–13%. These cases are likely influenced by comparatively limited training samples ( $n < 400$ ) and localized climatic variability. Despite these localized differences, the overall mean MAE improvement of approximately  $0.2464^\circ\text{C}$  across all districts remains substantial and statistically significant ( $p < 0.001$ ).

#### A. Limitations and Future Work

While results indicate clear one-day predictive gains, several limitations should be acknowledged:

- **Persistence-dominated signal:** One-day temperature forecasting benefits strongly from short-term autocorrelation; high  $R^2$  therefore partly reflects physical persistence rather than complex learned dynamics.
- **Data coverage gaps:** Although the dataset spans 2010–2024, some high-altitude districts and early years have reduced effective coverage; localized performance in low-sample districts is less certain.
- **Geographic encoding:** District label encoding is a compact proxy; it may insufficiently capture intra-district spatial heterogeneity. Future work should evaluate one-hot encodings or explicit lat/lon features.
- **Model horizon and complexity:** The present system focuses on next-day temperature. Extending to multi-day horizons will likely require sequence models (e.g., LSTMs or temporal convolutional networks) and richer predictors.
- **Extreme weather indicators:** While direct disaster prediction is outside the scope of the current system, future extensions could explore early-warning indicators for weather-driven hazards such as heatwaves, cold spells, heavy rainfall events, and flood risk estimation by combining precipitation forecasts with hydrological thresholds.
- **Ground-truth validation:** Integrating local meteorological station data and IoT observations would strengthen ground-truth validation and reduce reliance on reanalysis-only targets.

Future work will prioritize multi-day forecasting experiments, explicit spatial feature incorporation, and pilot integration of local station feeds where available.

WeatherWave demonstrates that localized machine learning models, when integrated with modern web architectures, can deliver accurate next-day temperature forecasts for geographically complex regions such as Nepal. By combining satellite-derived data, lightweight Random Forest regression, and a Progressive Web App interface, the system achieves statistically significant improvements over standard API-based persistence forecasting while remaining computationally efficient. Future work will extend the forecasting horizon, incorporate explicit spatial features, and explore extreme weather indicators using enriched ground-based observations.

#### REFERENCES

- [1] S. Scher and G. Messori, "Machine learning methods for weather forecasting," *J. Adv. Model. Earth Syst.*, 2019.
- [2] A. Giri and V. R. Giri, "Forecasting cauliflower prices in Nepal," *Cogent Food & Agriculture*, 2024.
- [3] J. Archibald, S. M. Barnes, and P. Kelly, "Progressive Web Applications," in *Proc. ICWE*, 2019.
- [4] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.